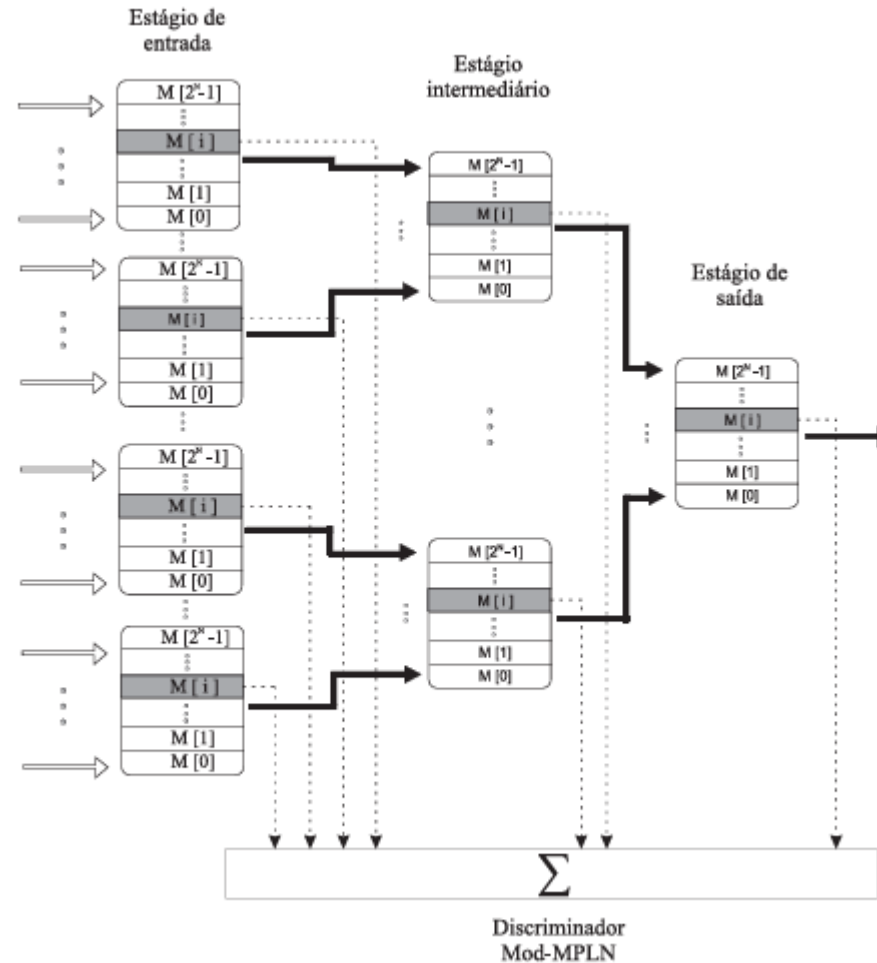


Inteligência Artificial

Rede Neural Artificial Sem Peso com Nó de Lógica Probabilística.

Rede Neural Sem Peso



Rede Neural Sem Peso

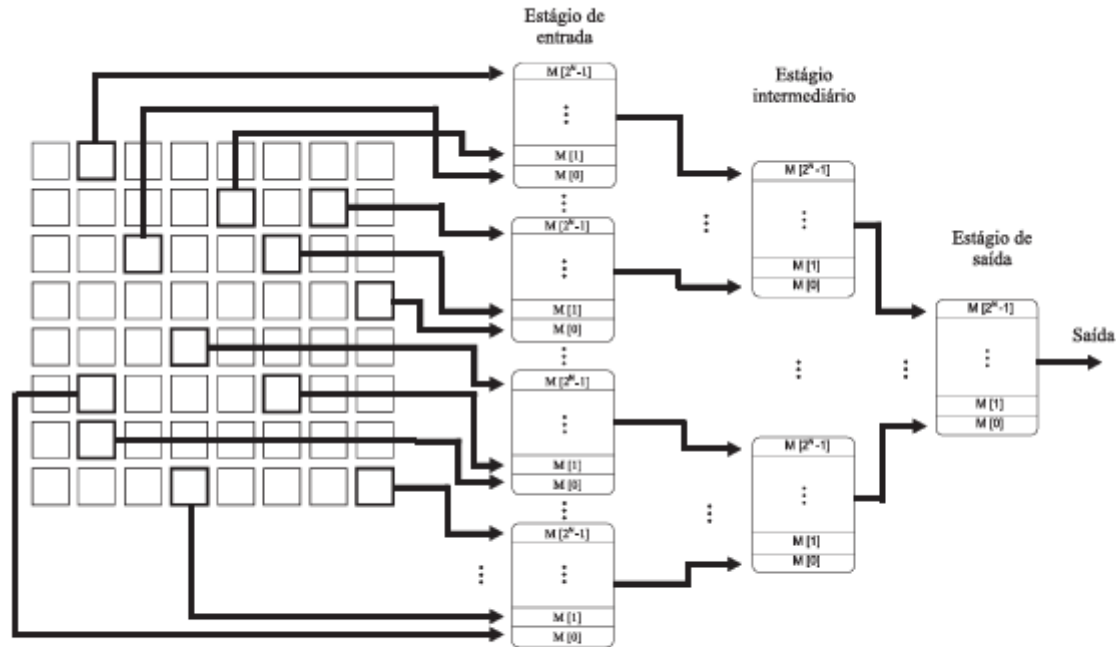


Figura : Estrutura de uma PLN

Rede Neural Sem Peso

- Nas aulas anteriores, vimos que uma **rede neural artificial convencional** (ex.: *perceptron*) **aprende ajustando pesos sinápticos por meio de regras de atualização (erro, gradiente etc.)**. O conhecimento fica distribuído numericamente nas conexões.
- As **Redes Neurais Sem Peso (RNSP)**, por outro lado, substituem o par (peso + função de ativação) por **nós de memória endereçável (nós RAM)**: cada neurônio é, na prática, uma **pequena memória binária**. O padrão de entrada (um vetor de bits) é usado diretamente como endereço de acesso a essa memória, e o valor armazenado naquele endereço é a saída do neurônio.
- O problema do nó RAM clássico é: **o que ele responde quando o endereço nunca foi visitado durante o treinamento?** É aqui que entra o **Nó Lógico Probabilístico (PLN)**: **em vez de responder sempre 0 (ou sempre indefinido) para endereços não treinados, o PLN sorteia a resposta com base em uma distribuição de probabilidade associada àquela posição de memória — permitindo generalização e simulando o comportamento de uma sinapse "incerta"**.

Rede Neural Sem Peso

Fundamentação teórica resumida

Um nó PLN possui, para cada posição de memória (endereço), **três estados possíveis**, em vez de apenas 0 ou 1:

Estado armazenado	Significado	Resposta do nó quando ativado
0	Endereço associado apenas à classe 0	Sempre responde 0
1	Endereço associado apenas à classe 1	Sempre responde 1
u (indefinido)	Endereço nunca treinado, ou treinado com ambas as classes (empate)	Sorteia 0 ou 1 com probabilidade p (geralmente $p = 0,5$)

Rede Neural Sem Peso

Regra de treinamento (simplificada para esta tarefa):

1. Para cada padrão de treino, calcula-se o endereço binário (concatenação dos bits de entrada atribuídos àquele nó).
2. Se a posição de memória está vazia, grava-se o rótulo da classe (0 ou 1).
3. Se a posição já contém um rótulo **diferente** do novo padrão, o conteúdo passa a **U** (indefinido/conflito).
4. Se a posição já contém o **mesmo** rótulo, nada muda.

Regra de teste:

- Calcula-se o endereço do padrão de teste.
- Se o conteúdo é **0** ou **1**, essa é a resposta do nó.
- Se o conteúdo é **U**, sorteia-se a resposta (na simulação manual, o aluno deve indicar as duas respostas possíveis e a probabilidade de cada uma).

Um **discriminador** RNSP-PLN é um conjunto de nós PLN que "votam"; a classe reconhecida é aquela com maior número de nós respondendo 1 (soma de respostas).

Rede Neural Sem Peso

- **Exemplo resolvido (passo a passo):**
- Considere um **discriminador** com **2 nós PLN**, cada um com **endereço de 2 bits** (logo, **4 posições de memória** por nó: **00, 01, 10, 11**).
- **Entrada:** vetor de 4 bits [**x1 x2 x3 x4**].
- **Nó A** lê os bits **x1 x2** (endereço A).
- **Nó B** lê os bits **x3 x4** (endereço B).

Rede Neural Sem Peso

Conjunto de treinamento

Padrão	x1 x2 x3 x4	Endereço A (x1x2)	Endereço B (x3x4)	Classe
P1	1 0 1 1	10	11	1
P2	1 0 0 1	10	01	1
P3	0 1 1 1	01	11	0
P4	1 0 1 1	10	11	1
P5	0 1 0 0	01	00	0

Rede Neural Sem Peso

- **Passo 1** — Preencher a memória do **Nó A** (endereços 00, 01, 10, 11):
- Endereço 10 recebe classe 1 (de P1), depois classe 1 de novo (P2) e classe 1 de novo (P4) → conteúdo final: 1.
- Endereço 01 recebe classe 0 (de P3), depois classe 0 (de P5) → conteúdo final: 0.
- Endereços 00 e 11 nunca foram treinados → conteúdo: u (de undefined).

Rede Neural Sem Peso

- **Passo 2** — Preencher a memória do **Nó B**:
- Endereço 11 recebe classe 1 (P1), depois classe 0 (P3) → conflito → conteúdo final: u.
- Endereço 01 recebe classe 1 (de P2) → conteúdo final: 1.
- Endereço 00 recebe classe 0 (de P5) → conteúdo final: 0.
- Endereço 10 nunca foi treinado → conteúdo: u.

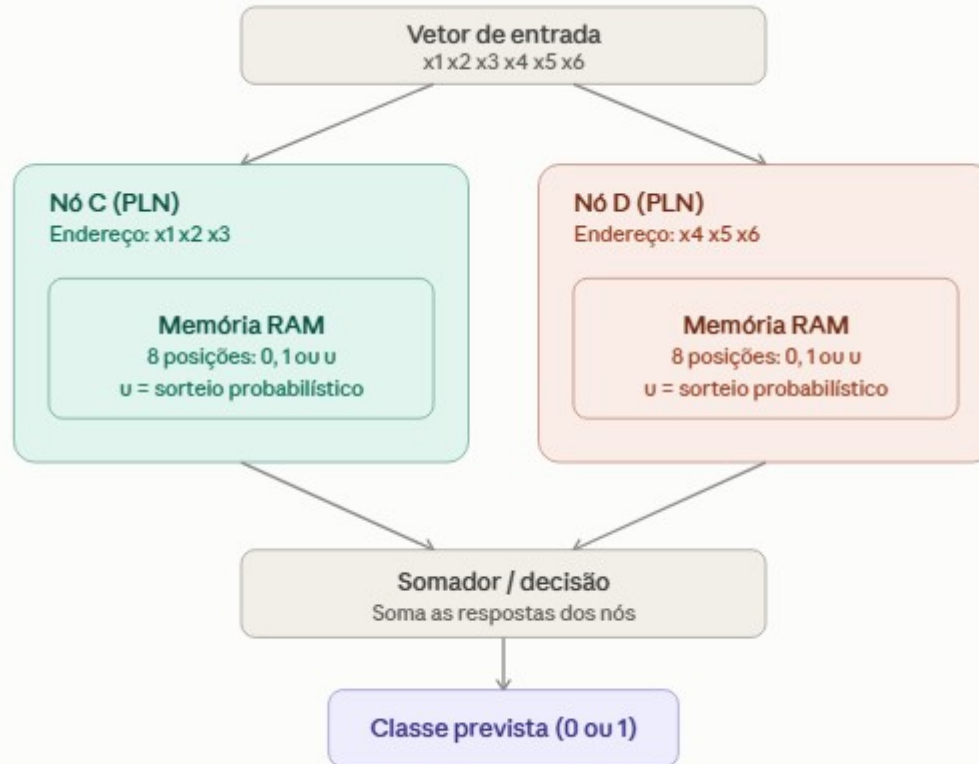
Rede Neural Sem Peso

- **Memória do Nó A:** $00 \rightarrow u$ | $01 \rightarrow 0$ | $10 \rightarrow 1$ | $11 \rightarrow u$
- **Memória do Nó B:** $00 \rightarrow 0$ | $01 \rightarrow 1$ | $10 \rightarrow u$ | $11 \rightarrow u$
- **Passo 3** — Teste com um novo padrão: $x = [1 \ 0 \ 1 \ 1]$ (idêntico a P1/P4)
- **Nó A:** endereço 10 → conteúdo 1 → resposta = 1
- **Nó B:** endereço 11 → conteúdo u → resposta = sorteio (0 ou 1, $p = 0,5$ cada)

Rede Neural Sem Peso

- **Resultado:** o discriminador responde com **certeza absoluta** pelo **Nó A** (**soma parcial = 1**) e incerteza pelo **Nó B**.
- **A saída final do discriminador é probabilística:** **soma = 1** (se B sortear 0) ou **soma = 2** (se B sortear 1) — em ambos os casos, o **padrão** tende a ser **reconhecido** como **classe 1**, mas o **grau de confiança** não é máximo, justamente porque um dos nós nunca "viu" bem esse endereço.

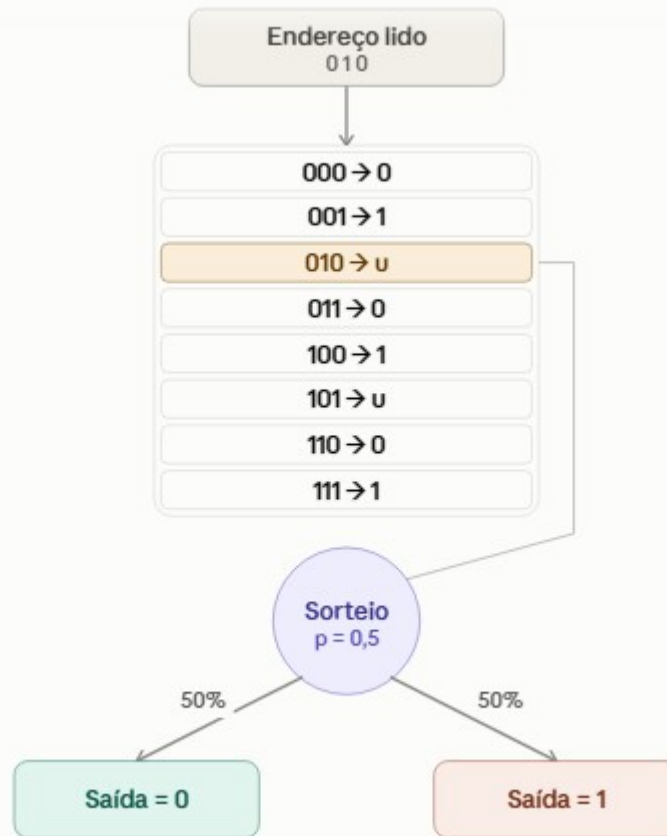
Rede Neural Sem Peso



Um outro Exemplo

- Um outro **discriminador**: o **vetor de 6 bits** é dividido entre o **Nó C** (lê **x1 x2 x3**) e o **Nó D** (lê **x4 x5 x6**), cada um com sua própria **memória RAM de 8 posições** (podendo assumir os **estados 0, 1 ou u**).
- As duas respostas seguem para o **somador/decisão**, que **soma os votos** e produz a **classe prevista** — sendo que, quando **algum nó está em estado u**, o **resultado final** passa a ter **caráter probabilístico**, como discutido no exemplo resolvido.

Um outro Exemplo



Um outro Exemplo

- Em binário, a paridade de um número depende exclusivamente do bit menos significativo (x_4):
 - se $x_4 = 0$, o número é par;
 - se $x_4 = 1$, é ímpar.
- Os bits x_1 , x_2 e x_3 são irrelevantes para essa decisão. Isso vai gerar um resultado pedagogicamente interessante, como você verá.

Um outro Exemplo

Arquitetura da rede

Igual ao padrão que já usamos: entrada de 4 bits `[x1 x2 x3 x4]`, dividida em 2 nós PLN com endereço de 2 bits cada:

- Nó E lê `x1 x2` (bits que nada têm a ver com paridade)
- Nó F lê `x3 x4` (inclui o bit determinante, `x4`)

Conjunto de treinamento

Padrão	x1 x2 x3 x4	x4 (LSB)	Classe (0=par, 1=ímpar)
T1	0 0 0 0	0	0
T2	0 0 1 1	1	1
T3	1 1 0 0	0	0
T4	1 1 1 1	1	1
T5	0 1 0 1	1	1
T6	1 0 1 0	0	0

Um outro Exemplo

Treinamento — Nó E (endereço x1x2)

Padrão	Endereço	Classe	O que acontece na memória
T1	00	0	grava 0 em 00
T2	00	1	00 já tinha 0, chega 1 → conflito → u
T3	11	0	grava 0 em 11
T4	11	1	11 já tinha 0, chega 1 → conflito → u
T5	01	1	grava 1 em 01
T6	10	0	grava 0 em 10

Memória final do Nó E: 00→u | 01→1 | 10→0 | 11→u

Um outro Exemplo

Treinamento — Nó F (endereço x3x4)

Padrão	Endereço	Classe	O que acontece na memória
T1	00	0	grava 0 em 00
T2	11	1	grava 1 em 11
T3	00	0	00 já era 0 → mantém
T4	11	1	11 já era 1 → mantém
T5	01	1	grava 1 em 01
T6	10	0	grava 0 em 10

Memória final do Nó F: 00→0 | 01→1 | 10→0 | 11→1

Um outro Exemplo

- Repare: o **Nó F** não teve nenhum conflito. Como seu endereço inclui x_4 (o bit que realmente decide a classe), toda vez que x_4 é o mesmo, a classe também é — mesmo variando x_3 . O **Nó E**, por outro lado, ficou com **metade das posições indefinidas**, porque x_1 e x_2 não carregam nenhuma informação sobre paridade.
- **Fase de teste**
Teste 1: $x = [1\ 0\ 0\ 1]$ (valor 9, ímpar $\rightarrow$ classe correta = 1)
- **Nó E:** endereço 10 $\rightarrow$ conteúdo 0 $\rightarrow$ responde 0 (errado, mas é "confiante")
- **Nó F:** endereço 01 $\rightarrow$ conteúdo 1 $\rightarrow$ responde 1 (correto)
- Soma dos votos: $0 + 1 = 1 \rightarrow$ **empate entre as duas classes possíveis** (1 nó votou 0, o outro votou 1)
- **Esse empate é resolvido da mesma forma que o estado u:** como não há maioria clara, o discriminador recorre a um critério probabilístico (ex.: 50%/50%).

Um outro Exemplo

- **Teste 2:** $x = [0\ 0\ 1\ 1]$ (valor 3, ímpar $\rightarrow$ classe correta = 1), idêntico a T2
- **Nó E:** endereço 00 $\rightarrow$ conteúdo u $\rightarrow$ sorteio (50% para cada lado)
- **Nó F:** endereço 11 $\rightarrow$ conteúdo 1 $\rightarrow$ responde 1 (correto, com certeza)
-
- **Aqui, mesmo com incerteza no Nó E, a resposta do Nó F "puxa" o resultado para a classe correta na maioria das rodadas de sorteio — mostrando como nós confiáveis compensam nós ruidosos.**

Um outro Exemplo

- Esse exemplo é ótimo para discutir um ponto que a arquitetura por si só não deixa óbvio: a qualidade do aprendizado de uma RNSP-PLN depende diretamente de como os bits de entrada são distribuídos entre os nós. Um nó que recebe bits irrelevantes para o problema tende a acumular conflitos (u) e a votar de forma não confiável; um nó que recebe o bit determinante aprende com perfeição.

Memorização ou Generalização

Efeito do tamanho do endereço

O tamanho do endereço de um nó RAM/PLN define quantas posições de memória ele possui (2^n posições para um endereço de n bits). Isso cria uma tensão direta entre **memorização** e **generalização**:

Endereço menor (poucos bits)	Endereço maior (muitos bits)
Poucas posições de memória → cada posição recebe muitos padrões de treino diferentes durante o treinamento	Muitas posições de memória → cada padrão de treino tende a cair numa posição "só sua"
Mais conflitos de classe na mesma posição → mais estados (U) por sobreposição de rótulos diferentes	Menos sobreposição, logo menos conflitos por esse motivo
Boa generalização: dois padrões parecidos tendem a compartilhar endereço e "puxar" a mesma resposta	Tendência à memorização: cada padrão de teste, se não for idêntico a algum de treino, cai numa posição nunca vista (U)
Risco de subajuste (<i>underfitting</i>): o nó fica "grosseiro" demais, perdendo capacidade discriminativa	Risco de sobreajuste (<i>overfitting</i>): a rede "decora" o conjunto de treino, mas responde com incerteza (U) para qualquer padrão novo

Memorização ou Generalização

- Ou seja: **crescer o endereço não é estritamente positivo**. No exemplo do Nó C/D que fizemos (3 bits, 8 posições), o crescimento reduziu conflitos entre A1/A2/etc., mas se aumentássemos para 6 bits (endereço = a entrada inteira), cada padrão de treino ocuparia uma posição exclusiva — o nó "**memorizaria**" o conjunto de treino perfeitamente, mas praticamente qualquer entrada de teste diferente cairia em u, e o discriminador perderia toda **capacidade de generalizar**.

Memorização ou Generalização

- **Efeito da composição do endereço**
- Isso é o que o exemplo par/ímpar acabou de ilustrar muito bem: não basta o tamanho — quais bits específicos compõem o endereço de cada nó importa tanto quanto, ou mais.
-
- O Nó F (bits x3x4, incluindo o bit determinante da paridade) teve memória perfeitamente consistente, sem nenhum u, porque seu endereço captura exatamente a informação relevante para a tarefa.
- O Nó E (bits x1x2, irrelevantes para paridade) acumulou conflitos e ficou com metade das posições em u, apesar de ter o mesmo tamanho de endereço que o Nó F.
- Isso mostra que **a generalização não depende só de "quantos bits", mas de se os bits escolhidos são informativos para a classe que se quer discriminar**. Um endereço maior composto majoritariamente por bits irrelevantes generaliza mal do mesmo jeito que um endereço pequeno — só que por um motivo diferente (conflito de rótulos, em vez de memorização exclusiva).

Resumindo ...

- **Endereço pequeno** → poucas posições → mistura muitos padrões → risco de subajuste.
- **Endereço grande** → muitas posições → isola cada padrão → risco de sobreajuste (memorização).
- **Ponto ideal** → depende de quantos bits realmente carregam informação discriminativa para o problema.
- Isso conecta diretamente com o que se viu em redes com peso: é análogo à escolha da capacidade do modelo (número de neurônios/camadas) e à seleção de features relevantes — só que aqui o "ajuste de capacidade" é feito escolhendo o tamanho e a composição do endereço de cada nó, em vez de ajustar pesos.

Um Caso Interessante

- Vamos simular um **discriminador** que aponta se um paciente tem quadro "**sugestivo de gripe**" (1) ou "**não sugestivo**" (0), a partir de **4 sintomas binários** (presente = 1, ausente = 0):
 - x_1 = febre
 - x_2 = tosse
 - x_3 = dor de cabeça
 - x_4 = dor muscular
- Para fins didáticos, a regra "real" (que nem todos conhecem de antemão — é o que a rede deve descobrir implicitamente) é: o quadro é sugestivo de gripe sempre que há febre ($x_1 = 1$), independentemente dos demais sintomas.

Um Caso Interessante

Arquitetura

Mesma estrutura de sempre: entrada de 4 bits dividida em 2 nós PLN de endereço 2 bits.

- Nó G lê **x1 x2** (febre, tosse)
- Nó H lê **x3 x4** (dor de cabeça, dor muscular)

Conjunto de treinamento

Paciente	x1 x2 x3 x4	Febre?	Classe (1=sugere gripe)
P1	1 1 0 0	sim	1
P2	1 1 1 1	sim	1
P3	0 0 1 1	não	0
P4	0 1 1 0	não	0
P5	1 0 0 1	sim	1
P6	0 0 0 0	não	0

Um Caso Interessante

Treinamento — Nó G (endereço x1x2)

Paciente	Endereço	Classe	Efeito na memória
P1	11	1	grava 1
P2	11	1	já era 1 → mantém
P3	00	0	grava 0
P4	01	0	grava 0
P5	10	1	grava 1
P6	00	0	já era 0 → mantém

Memória do Nó G: 00→0 | 01→0 | 10→1 | 11→1 — sem nenhum conflito.

Um Caso Interessante

Treinamento — Nó H (endereço x3x4)

Paciente	Endereço	Classe	Efeito na memória
P1	00	1	grava 1
P2	11	1	grava 1
P3	11	0	11 já era 1, chega 0 → conflito → u
P4	10	0	grava 0
P5	01	1	grava 1
P6	00	0	00 já era 1, chega 0 → conflito → u

Memória do Nó H: 00→u | 01→1 | 10→0 | 11→u

Um Caso Interessante

Note o padrão: o **Nó G**, que inclui a febre (bit realmente determinante), ficou perfeitamente consistente. O **Nó H**, que só enxerga sintomas inespecíficos (dor de cabeça e dor muscular, que aparecem tanto em quadros gripais quanto em outras causas), acumulou dois conflitos.

Fase de teste

Teste 1: paciente com $x = [1 \ 0 \ 1 \ 1]$ (febre, sem tosse, com dor de cabeça e dor muscular)

- **Nó G:** endereço 10 → conteúdo 1 → responde **1**, com certeza
- **Nó H:** endereço 11 → conteúdo u → **sorteio (50%/50%)**
- **Resultado:** mesmo com a incerteza do **Nó H**, o voto confiante do **Nó G** empurra o discriminador para "sugestivo de gripe" — coerente com o fato de haver febre.

Teste 2: paciente com $x = [0 \ 1 \ 1 \ 0]$ (sem febre, com tosse e dor de cabeça)

- **Nó G:** endereço 01 → conteúdo 0 → responde **0**, com certeza
- **Nó H:** endereço 10 → conteúdo 0 → responde **0**, com certeza
- **Resultado:** ambos os nós concordam com certeza → "não sugestivo de gripe", sem qualquer incerteza no resultado.

Rede Neural Sem Peso MPLN

- A **RNSP MPLN** é a extensão natural do **PLN**.
- O **MPLN** é uma generalização do **PLN**, também baseada em **memória RAM por nó**, mas que resolve uma limitação específica do **PLN clássico**: a **granularidade dos estados de memória**.

Rede Neural Sem Peso MPLN

Aspecto	PLN (o que estudamos)	MPLN
Estados possíveis por posição de memória	3: 0, 1, u (indefinido)	Múltiplos valores discretos, representando graus de probabilidade entre 0 e 1 (ex.: 0, 0.25, 0.5, 0.75, 1)
Resposta quando incerto	Sempre sorteio 50%/50%, sem meio-termo	Sorteio ponderado pelo grau armazenado — o nó pode estar "quase certo de 1" (ex.: 0.8) e não apenas "totalmente incerto"
Regra de atualização da memória	Regra simples de gravação/conflito (grava rótulo; se colide, vira u)	Regra de reforço (reward/penalty) : a cada apresentação de um padrão, o valor probabilístico armazenado é incrementado ou decrementado gradualmente em direção à classe reforçada
Sensibilidade a ruído/repetição	Um único padrão conflitante já "zera" a posição para u, perdendo toda a informação anterior	Um padrão isolado ou ruidoso desloca o valor probabilístico só um pouco — a memória "acumula evidência" ao longo de várias apresentações
Capacidade de generalização	Binária: ou a posição "sabe", ou não sabe nada (u)	Gradual: a posição pode refletir o quão majoritária foi uma classe naquele endereço, mesmo com exceções

Rede Neural Sem Peso MPLN

Por que isso importa

No nosso exemplo do diagnóstico, se o Nó H (dor de cabeça + dor muscular) tivesse sido implementado como MPLN em vez de PLN:

- Em vez de simplesmente virar **u** diante do primeiro conflito (P3 chegando com classe 0 depois de já ter gravado 1), a posição **11** teria seu valor probabilístico apenas empurrado um pouco na direção de 0 — por exemplo, de "1,0" para "0,7" — preservando a maioria estatística observada, em vez de descartar toda a informação com um único conflito.
- Isso torna o MPLN mais **robusto a ruído** e mais próximo, em espírito, do ajuste incremental que os alunos já conhecem do gradiente descendente nas redes com peso: a cada exemplo, o modelo se move um pouco na direção do erro, em vez de "tudo ou nada".

Rede Neural Sem Peso MPLN

Como o MPLN representa a memória

Cada posição de memória do MPLN não guarda mais 0, 1 ou u, e sim um **contador discreto** que se traduz numa probabilidade de saída = 1. Vamos usar **5 níveis** (contador de 0 a 4, $N = 4$), começando todos em estado neutro:

$$p = \frac{\text{contador}}{N}, \quad \text{contador inicial} = N/2 = 2 \quad (p = 0,5)$$

Regra de treinamento (reforço/punição):

- Se o padrão de treino tem classe **1** → incrementa o contador em 1 (até o máximo, 4)
- Se o padrão de treino tem classe **0** → decrementa o contador em 1 (até o mínimo, 0)

Repare que, com $N = 2$ (apenas 3 níveis: 0, 1, 2 → $p = 0, 0,5, 1$), o MPLN se reduz **exatamente** ao comportamento do PLN que já estudamos (0, u, 1). O PLN é, portanto, um caso particular do MPLN com o menor número de níveis possível — vale destacar isso para a turma.

Rede Neural Sem Peso MPLN

Treinamento passo a passo — Nó H (endereço x3x4), N = 4

Estado inicial: 00→2 | 01→2 | 10→2 | 11→2 (todos p = 0,5)

Paciente	Endereço	Classe	Ação	Contador após
P1	00	1	incrementa 00	00: 2→3 (p=0,75)
P2	11	1	incrementa 11	11: 2→3 (p=0,75)
P3	11	0	decrementa 11	11: 3→2 (p=0,50)
P4	10	0	decrementa 10	10: 2→1 (p=0,25)
P5	01	1	incrementa 01	01: 2→3 (p=0,75)
P6	00	0	decrementa 00	00: 3→2 (p=0,50)

Memória final do Nó H (MPLN): 00→0,50 | 01→0,75 | 10→0,25 | 11→0,50

Rede Neural Sem Peso MPLN

Comparando com o PLN nos mesmos dados

Endereço	PLN (visto antes)	MPLN (agora)
00	u (sorteio 50/50)	$p = 0,50$ (sorteio 50/50 — mesmo resultado)
01	1 (certeza total)	$p = 0,75$ (tende a 1, mas não é absoluto)
10	0 (certeza total)	$p = 0,25$ (tende a 0, mas não é absoluto)
11	u (sorteio 50/50)	$p = 0,50$ (sorteio 50/50 — mesmo resultado)

Nos endereços 00 e 11, onde houve exatamente um exemplo de cada classe, os dois modelos convergem para a mesma incerteza. Mas nos endereços 01 e 10, que tiveram apenas um único exemplo de treino, o PLN já cravou certeza absoluta (0 ou 1), enquanto o MPLN foi mais cauteloso: um único exemplo desloca a probabilidade, mas não basta para "fechar" a decisão. Isso é exatamente o comportamento de acúmulo gradual de evidência que discutimos como vantagem do MPLN.

Rede Neural Sem Peso MPLN

Reforçando a evidência (mostrando a convergência para certeza)

Suponha que chegam mais dois pacientes:

Paciente	Endereço	Classe	Ação	Contador após
P7	10	0	decrementa 10	10: 1→0 ($p=0,00 \rightarrow$ agora sim, certeza)
P8	01	1	incrementa 01	01: 3→4 ($p=1,00 \rightarrow$ agora sim, certeza)

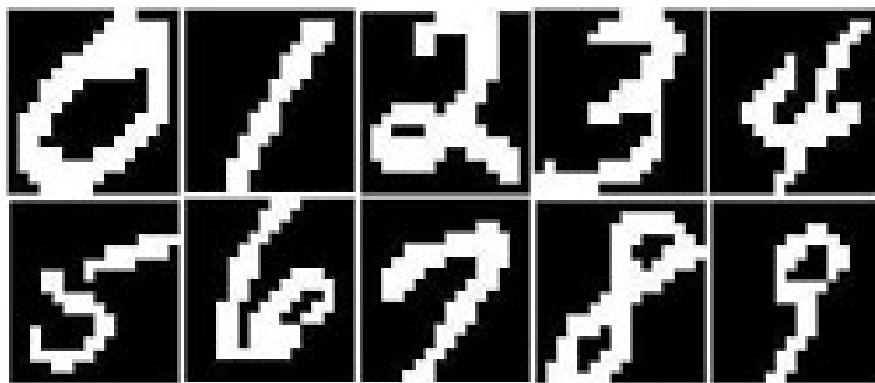
Só depois de **dois exemplos consistentes**, e não um só, o MPLN atinge o mesmo grau de certeza que o PLN atingiu de imediato. Esse é o ponto pedagógico central: o MPLN troca a "decisão precipitada" do PLN por uma convergência mais lenta e mais resistente a exemplos isolados ou ruidosos.

Teste com o novo comportamento

Paciente de teste $x = [0 \ 1 \ 1 \ 0]$ (mesmo do Teste 2 anterior) $\rightarrow$ endereço do Nó H = 10

- No **PLN**: memória continha 0 $\rightarrow$ resposta certa e determinística: 0
- No **MPLN**, antes de P7/P8: memória continha $p = 0,25 \rightarrow$ o nó responde **0 com 75% de chance** e **1 com 25% de chance** — ainda tende à resposta certa, mas reconhece que a evidência (um único caso) não era conclusiva
- No **MPLN**, depois de P7: $p = 0,00 \rightarrow$ resposta determinística **0**, agora com o mesmo nível de certeza do PLN, mas alcançada de forma mais criteriosa

Rede Neural Sem Peso MPLN



Com múltiplas Camadas

A definição original já permite múltiplas camadas

Na formulação clássica, uma rede PLN é definida como **um arranjo de um número finito de neurônios organizados em qualquer número de camadas, sendo cada neurônio um nó PLN**. Ou seja, camadas múltiplas fazem parte da própria definição do modelo, não são uma extensão posterior. O mesmo vale, por extensão, para o MPLN.

Mas o que simulamos até aqui é, na prática, de uma camada só

Nos exemplos que construímos (nós C/D, E/F, G/H), tínhamos **uma única camada de nós PLN**, cujas saídas iam diretamente para o somador de decisão. Essa é, de fato, a arquitetura mais usada na prática — o modelo **WiSARD** (o mais famoso sistema de reconhecimento de padrões baseado em RNSP) funciona assim: um discriminador por classe, cada um com uma única camada de nós RAM/PLN votando. Não há "camada oculta" verdadeira.

Com múltiplas Camadas

Onde aparecem camadas de fato — arquiteturas piramidais

Existem arquiteturas **multicamadas** documentadas na literatura, chamadas de **redes piramidais** (pyramidal WNN). A ideia:

- **Camada 1:** muitos nós PLN pequenos, cada um lendo uma região local da entrada (ex.: um pedaço da imagem) — igual ao que fizemos, mas em escala menor e paralela.
- **Camada 2:** um novo conjunto de nós PLN cujo **endereço não é mais formado pelos bits de entrada originais, e sim pelas saídas (0/1) dos nós da camada 1.**
- Isso se repete por mais camadas, cada uma "comprimindo" a informação da anterior — daí o nome pirâmide: a quantidade de nós diminui a cada camada, até chegar a uma decisão final.

Essa estrutura é usada sobretudo em reconhecimento de padrões visuais (imagens maiores, onde uma única camada de nós teria endereços grandes demais e generalizaria mal, ligando de volta à nossa discussão sobre tamanho do endereço).

Com múltiplas Camadas

O problema que isso traz — e por que é pedagogicamente rico

Aqui está o ponto mais interessante para levar à turma: em uma MLP com peso, o erro da camada de saída é retropropagado até as camadas ocultas via gradiente (backpropagation), porque a função de cada neurônio é diferenciável. **Um nó RAM/PLN não é uma função diferenciável — é uma tabela de consulta.** Não existe "gradiente" para propagar.

Por isso, treinar uma RNSP-PLN/MPLN multicamada exige estratégias diferentes das que os alunos já conhecem, por exemplo:

- **Treinamento por reforço global** (reward/penalty), que é a mesma lógica que já vimos no MPLN, só que aplicada também aos nós das camadas intermediárias, mesmo sem saber exatamente "de quem foi a culpa" pelo erro (isso é chamado de **problema de atribuição de crédito**, credit assignment).
- **Camadas intermediárias fixas/aleatórias**, onde só a última camada é de fato treinada — abordagem que lembra bastante os modelos de projeção aleatória (ex.: Extreme Learning Machines) que talvez valha citar como analogia.

Com múltiplas Camadas

- Uma RNSP-PLN/MPLN de uma camada só (o que simulamos) é simples e rápida, mas seu poder de representação é limitado pelo tamanho do endereço de cada nó.
- Empilhar camadas resolve isso — mas troca o problema de "generalização por endereço" pelo problema de "como treinar sem gradiente", que é conceitualmente análogo ao que motivou o desenvolvimento do backpropagation nas redes com peso.

Com múltiplas Camadas

O problema

Entrada de 6 bits $[x_1 \ x_2 \ x_3 \ x_4 \ x_5 \ x_6]$. A classe correta é definida em duas etapas (propositalmente, para exigir 2 camadas):

1. Calcule se a **primeira metade** ($x_1x_2x_3$) tem maioria de bits 1, e o mesmo para a **segunda metade** ($x_4x_5x_6$).
2. A classe final é o **XOR** dessas duas maiorias (1 se as metades "discordam", 0 se "concordam").

Um único nó não resolveria isso sozinho de forma direta — é exatamente o tipo de combinação não linear que justifica empilhar uma segunda camada.

Exemplo do XOR

Camada 1 — Nó C (endereço x1x2x3) e Nó D (endereço x4x5x6)

Como o endereço de 3 bits já contém toda a informação necessária (a maioria é função determinística do próprio endereço), a memória, uma vez totalmente treinada, fica assim para ambos os nós:

Endereço	Qtde de 1s	Maioria	Conteúdo da memória
000	0	0	0
001	1	0	0
010	1	0	0
100	1	0	0
011	2	1	1
101	2	1	1
110	2	1	1
111	3	1	1

Memória do Nó C = Memória do Nó D (mesma tabela, aplicada a metades diferentes da entrada).

Exemplo do XOR

Camada 2 — Nó Z (endereço = saídas de C e D)

O Nó Z não lê mais bits da entrada original: seu endereço de 2 bits é formado pelas saídas da camada 1, `[saída_C saída_D]`. O alvo é o XOR:

Endereço (C,D)	Classe (XOR)	Conteúdo da memória
00	0	0
01	1	1
10	1	1
11	0	0

Sem conflitos — cada combinação (C,D) aparece uma única vez no treinamento.

Exemplo do XOR

Exemplo 1 — caso totalmente determinístico

Entrada de teste: $x = [1 \ 1 \ 0 \ 0 \ 0 \ 1]$

Camada 1:

- Nó C lê $x_1x_2x_3 = 110 \rightarrow$ memória diz $1 \rightarrow$ saída $C = 1$
- Nó D lê $x_4x_5x_6 = 001 \rightarrow$ memória diz $0 \rightarrow$ saída $D = 0$

Camada 2:

- Nó Z lê o endereço $[C \ D] = 10 \rightarrow$ memória diz $1 \rightarrow$ classe final = 1

Exemplo do XOR

Exemplo 2 — outro caso determinístico

Entrada de teste: $x = [0 \ 1 \ 1 \ 1 \ 0 \ 1]$

Camada 1:

- Nó C lê $011 \rightarrow$ saída $C = 1$
- Nó D lê $101 \rightarrow$ saída $D = 1$

Camada 2:

- Nó Z lê $[C \ D] = 11 \rightarrow$ memória diz $0 \rightarrow$ classe final = 0

Conferindo: as duas metades têm maioria 1 $\rightarrow$ "concordam" $\rightarrow$ XOR = 0. ✓

Exemplo do XOR

Exemplo 3 — o caso interessante: incerteza propagando entre camadas

Agora suponha que, durante o treinamento, o Nó C **nunca recebeu nenhum exemplo com endereço 101** (por falha de amostragem, não por impossibilidade lógica). Sua memória fica assim nessa posição: $101 \rightarrow u$.

Entrada de teste: $x = [1 \ 0 \ 1 \ 1 \ 1 \ 1]$

Camada 1:

- Nó C lê $x_1x_2x_3 = 101 \rightarrow$ memória diz $u \rightarrow$ sorteio: $C = 0$ com $p=0,5$ ou $C = 1$ com $p=0,5$
- Nó D lê $x_4x_5x_6 = 111 \rightarrow$ memória diz $1 \rightarrow$ saída $D = 1$, com certeza

Camada 2 — dois cenários possíveis, dependendo do sorteio da camada 1:

Sorteio de C	Endereço de Z	Saída de Z	Probabilidade deste caminho
$C = 0$ (50%)	$[0,1] = 01$	1	0,5
$C = 1$ (50%)	$[1,1] = 11$	0	0,5

Resultado final: classe = 1 com 50% de chance, classe = 0 com 50% de chance — mesmo o Nó Z sendo totalmente determinístico e sem nenhum conflito próprio!

Exemplo do XOR